---------- Slip 12 ----------

Q.1) Write a C program that print the exit status of a terminated child process

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    pid_t pid;
    int status;

    // Create a child process
    pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        // Child process
        printf("Child process PID: %d\n", getpid());
        sleep(2); // Simulate some work
        exit(42); // Child exits with status 42
    } else {
        // Parent process
        wait(&status); // Wait for child to terminate

        if (WIFEXITED(status)) {
            printf("Child exited normally with status: %d\n", WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("Child was terminated by signal: %d\n", WTERMSIG(status));
        } else {
            printf("Child terminated abnormally.\n");
```

```
        }
    }

    return 0;
}
```

Q.2) Write a C program which receives file names as command line arguments and display those filenames in ascending order according to their sizes. I) (e.g $ a.out a.txt b.txt c.txt, ...)

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <string.h>

// Structure to hold file name and size
typedef struct {
    char *name;
    off_t size;
} FileInfo;

int compare(const void *a, const void *b) {
    FileInfo *f1 = (FileInfo *)a;
    FileInfo *f2 = (FileInfo *)b;

    if (f1->size < f2->size) return -1;
    else if (f1->size > f2->size) return 1;
    else return 0;
}
int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <file1> <file2> ...\n", argv[0]);
        return 1;
    }

    int n = argc - 1;
```

```
    FileInfo *files = malloc(n * sizeof(FileInfo));
    if (!files) {
        perror("malloc");
        return 1;
    }

    // Get file sizes
    for (int i = 0; i < n; i++) {
        struct stat st;
        if (stat(argv[i + 1], &st) < 0) {
            perror(argv[i + 1]);
            files[i].name = argv[i + 1];
            files[i].size = -1; // mark error
        } else {
            files[i].name = argv[i + 1];
            files[i].size = st.st_size;
        }
    }

    // Sort files by size
    qsort(files, n, sizeof(FileInfo), compare);

    // Display sorted files
    printf("Files in ascending order of size:\n");
    for (int i = 0; i < n; i++) {
        if (files[i].size >= 0)
            printf("%s\t%ld bytes\n", files[i].name, files[i].size);
        else
            printf("%s\tError getting size\n", files[i].name);
    }

    free(files);
    return 0;
}
```